

Assignment 15.4

Name: Navya Revuri

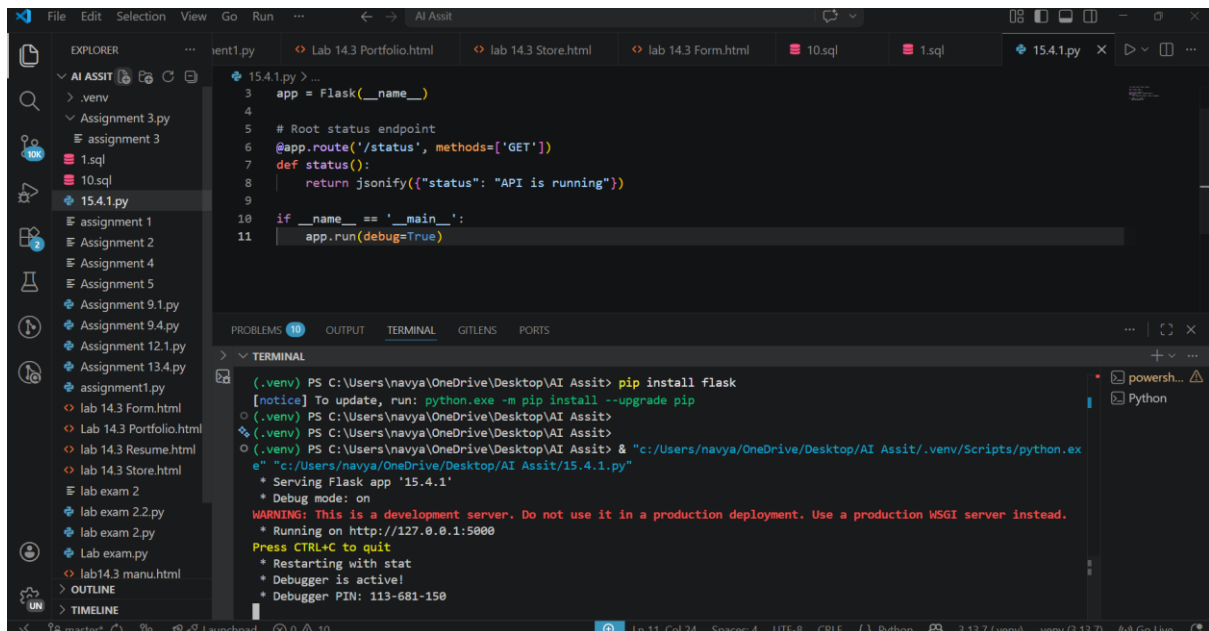
HTNO: 2303A52483

Batch 44

Task 1:

Prompt:

Generate a basic Flask application with a /status endpoint that returns JSON {"status": "API is running"}.

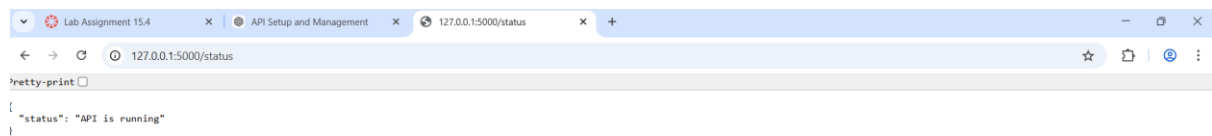


The screenshot shows a Visual Studio Code editor with a project named 'AI Assit'. The Explorer sidebar on the left shows a file tree with folders like '.venv', 'Assignment 3', and 'Assignment 1'. The main editor window displays the file '15.4.1.py' with the following Python code:

```
1 15.4.1.py > ...
2
3  app = Flask(__name__)
4
5  # Root status endpoint
6  @app.route('/status', methods=['GET'])
7  def status():
8      return jsonify({"status": "API is running"})
9
10 if __name__ == '__main__':
11     app.run(debug=True)
```

Below the code editor, the TERMINAL panel shows the execution of the application:

```
> TERMINAL
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit> pip install flask
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit>
(.venv) PS C:\Users\navya\OneDrive\Desktop\AI Assit> & "c:/Users/navya/OneDrive/Desktop/AI Assit/.venv/Scripts/python.exe"
e" "c:/Users/navya/OneDrive/Desktop/AI Assit/15.4.1.py"
 * Serving Flask app "15.4.1"
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 113-681-150
```



Observation:

- Flask server started successfully.
- Accessing /status returned:

```
{"status": "API is running"}
```

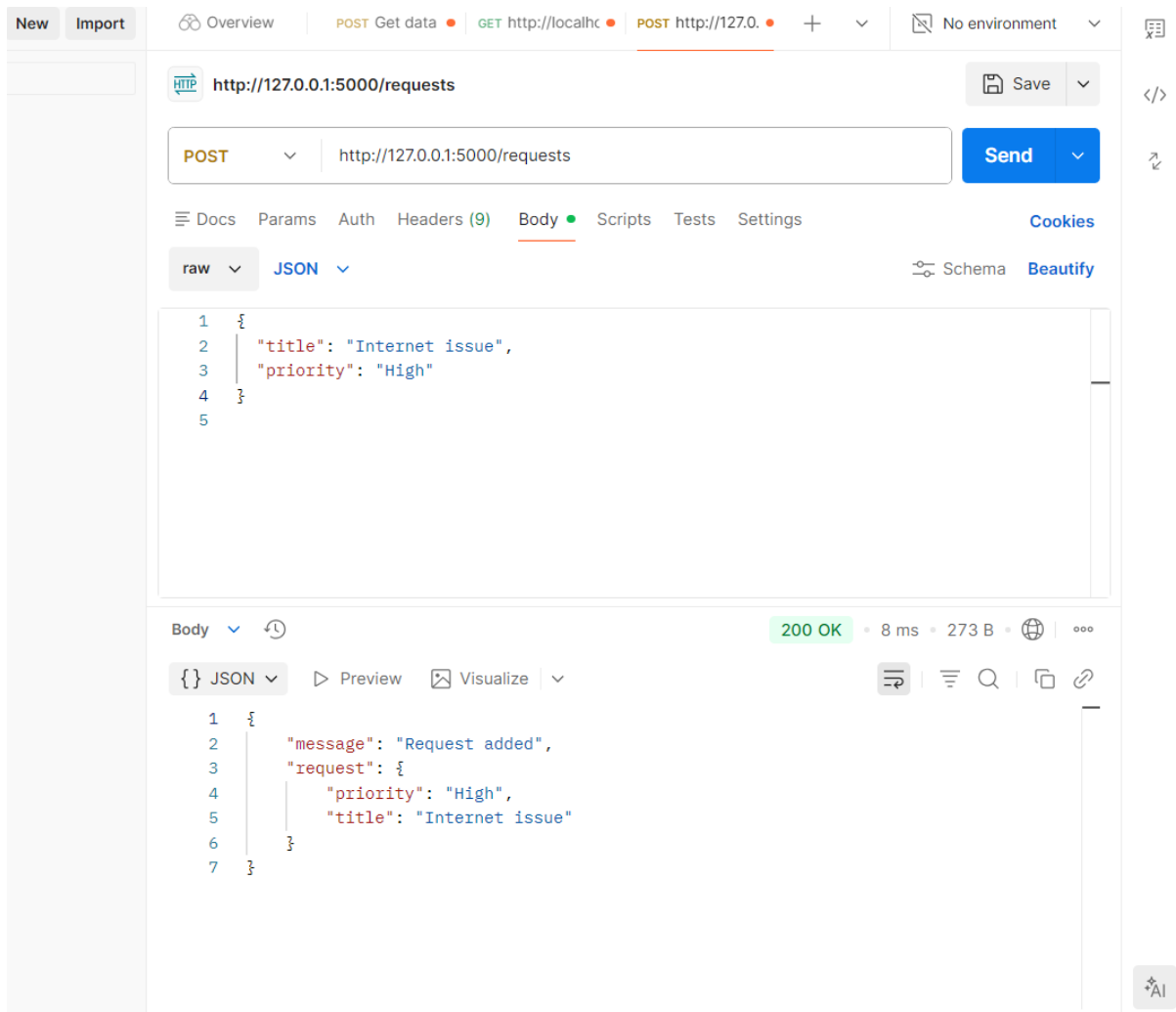
- Confirms backend is working.

Task 2:

Prompt: Create GET and POST endpoints in Flask to store service requests in memory. Each request has title and priority.

```
15.4.2.py > ...
1  #Create GET and POST endpoints in Flask to store service requests in
2  from flask import Flask, jsonify, request
3  app = Flask(__name__)
4  # In-memory storage
5  requests_data = []
6  # GET all requests
7  @app.route('/requests', methods=['GET'])
8  def get_requests():
9      return jsonify(requests_data)
10 # POST new request
11 @app.route('/requests', methods=['POST'])
12 def add_request():
13     data = request.get_json()
14
15     new_request = {
16         "title": data.get("title"),
17         "priority": data.get("priority")
18     }
19
20     requests_data.append(new_request)
21
22     return jsonify({
23         "message": "Request added",
24         "request": new_request
25     })
26
27 if __name__ == '__main__':
28     app.run(debug=True)
```

← → ↻ 127.0.0.1:5000/requests
Pretty-print ☐
[]



Observation :

- Initially GET returned empty list
- POST successfully added request
- Data stored temporarily in memory
- **Subsequent GET returned added data**

Task 3:

Prompt: Add a PUT endpoint to update a service request by index with proper error handling.

```
lab 14.3 Store.html lab 14.3 Form.html 10.sql 1.sql 15.4.1.py 15.4.2.py
15.4.3.py > update_request
1 # PUT update request
2 @app.route('/requests/<int:index>', methods=['PUT'])
3 def update_request(index):
4
5     if index < 0 or index >= len(requests_data):
6         return jsonify({"error": "Request not found"}), 404
7
8     data = request.get_json()
9
10    requests_data[index]["title"] = data.get("title", requests_data[index]["title"])
11    requests_data[index]["priority"] = data.get("priority", requests_data[index]["priority"])
12
13    return jsonify({
14        "message": "Request updated",
15        "request": requests_data[index]
16    })
```

 http://127.0.0.1:5000/requests/0  Save

PUT

http://127.0.0.1:5000/requests/0

Send

Docs Params Auth Headers (9) Body Scripts Tests Settings

Cookies

raw

JSON

Schema Beautify

```
1 {
2   | "priority": "Low"
3   | }
4
```

Body

404 NOT FOUND • 7 ms • 388 B •  

HTML

Preview

Debug with AI

```
1 <!doctype html>
2 <html lang=en>
3 <title>404 Not Found</title>
4 <h1>Not Found</h1>
5 <p>The requested URL was not found on the server. If you entered the URL manually
  | please check your spelling and try
6   again.</p>
```

◆ Observation

- Valid index updated successfully
- Returned updated object
- Invalid index returned:

```
{"error": "Request not found"}
```

- HTTP status: 404

Task 4:

Prompt:

Create a DELETE endpoint to remove a request by index and return deleted data.

```
15.4.4.py > delete_request
1  # ◆ DELETE request
2  @app.route('/requests/<int:index>', methods=['DELETE'])
3  def delete_request(index):
4
5      if index < 0 or index >= len(requests_data):
6          return jsonify({"error": "Request not found"}), 404
7
8      deleted = requests_data.pop(index)
9
10  return jsonify({ import jsonify
11                    "message": "Request deleted",
12                    "request": deleted
13                })
```

The screenshot shows a web client interface for a REST API. The URL bar displays `http://127.0.0.1:5000/requests/0`. The method is set to `DELETE`. The request body is a JSON object: `{ "priority": "Low" }`. The response status is `404 NOT FOUND` with a response time of 7 ms and a size of 388 B. The response body is an HTML document with the following content:

```
<!doctype html>
<html lang=en>
<title>404 Not Found</title>
<h1>Not Found</h1>
<p>The requested URL was not found on the server. If you entered the URL manually
please check your spelling and try
again.</p>
```

Observation

- Request removed successfully
- Deleted data returned
- Invalid index handled safely
- Server did not crash

Task 5:

Prompt:

Add docstrings and comments to Flask API endpoints for better readability and documentation.

```
15.4.5.py > get_requests
1  @app.route('/requests', methods=['GET'])
2  def get_requests():
3      """
4      GET /requests
5      Returns all service requests.
6
7      Response:
8      [
9          {
10             "title": "string",
11             "priority": "string"
12         }
13     ]
14     """
15     return jsonify(requests_data)
```


POST

▼

http://127.0.0.1:5000/requests/0

☰ Docs

Params

Auth

Headers (9)

Body ●

Scripts

Tests

Settings

raw ▼

JSON ▼

1

```
1 @app.route('/requests', methods=['POST'])
2 def add_request():
3     """
4     POST /requests
5     Adds a new service request.
6
7     Input:
8     {
9         "title": "string",
10        "priority": "string"
11    }
12
13    Response:
14    {
15        "message": "Request added",
16        "request": {...}
17    }
18    """
19
```

Observation

- Added docstrings for all endpoints
- Improved readability
- API easy for other developers to understand
- Swagger UI (optional) provides interactive documentation